Singly Linked List – GetNthNodeFromEnd 2 – Space Complexity and Algorithm

[*Nth Node from end of a Linked List*, 0 *based indexing*]

Program :

```
int getNthNodeFromEnd2(Node **head, int node)
{
    int len = 0;
    Node *current;
    current = *head;
    while (current != nullptr)
    {
        current = current->next;
        len++;
    }
    if (node < 0 || node >= len)
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
    else
    {
        current = *head;
        for (int i = 0; i < len - node - 1; i++)
        {
            current = current->next;
        }
        return current->data;
    }
}
case 21:
        {
            cout << endl;
            cout << endl;
```

```
            cout << "Nth node from end , where 0 is the last node(0 based
indexing):" << endl;
            int n;
            cout << "Enter the value of node: ";
            cin >> n;
            cout << "The " << n << "th node from the end is: " <<
getNthNodeFromEnd2(&head, n) << endl;
            cout << endl;
            cout << endl;
            break;
        }
```

Space Complexity Analysis

1. Input Space Complexity

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, O(1).***
- ***An integer (int node), which also requires a constant amount of space, O(1).***

Thus, the total input space complexity is: O(1).

• Node **head – one pointer-to-pointer (machine word)

• int node – one scalar (machine word)

Two fixed-size words → constant memory regardless of list length.

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int node` (an integer), both of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

$B. Full - data\ (inclusive) Definition$:

Add everything reachable from the input pointers at the moment the call starts.

*head may point to a singly linked list containing n nodes.

Those n Node objects dominate, so the total input footprint is Θ(n).

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to n. Therefore, the input space complexity is **Θ(n)**.

$2. Auxiliary\ Space\ Complexity$

$Auxiliary\ space\ complexity\ refers\ to\ the\ extra\ space\ used\ by\ the\ algorithm\ that\ is\ not\ part\ of\ the\ input. In\ the\ case\ of\ the\ getNthNodeFromEnd\ function, the\ auxiliary\ space\ is\ determined\ by\ the\ variables\ declared\ within\ the\ function$:

- $The\ function\ uses\ a\ pointer\ Node\ *current\ to\ traverse\ the\ linked\ list, which\ requires\ O(1) space.$

- $An\ integer\ variable\ int\ len\ is\ used\ to\ keep\ track\ of\ the\ current\ index\ during\ traversal, which\ also\ requires\ O(1)\ space.$

- $An\ integer\ variable\ `int\ i\ `used\ in\ for\ loop\ goes\ from\ 0\ to\ (len - node - 1) - 1, which\ actually\ takes\ constant\ amount\ of\ space\ i.e.\ O(1)\ space.$

$Since\ both\ of\ these\ variables\ occupy\ a\ constant\ amount\ of\ space\ regardless\ of\ the\ size\ of\ the\ linked\ list, the\ auxiliary\ space\ complexity\ is: O(1).$

What the routine itself allocates in addition to its input:

• int len – one scalar

• Node *current– one pointer

• A few compiler-generated temporaries / return address

No recursion and no dynamic heap allocations. The variables are reused for both passes over the list, so the additional memory never grows with n.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. `int len`: The function declares a single integer to store the list's length. An integer occupies a constant amount of memory.
2. `Node *current`: It declares a single pointer to traverse the list. This pointer also occupies a constant amount of memory and is reused for both passes.
3. `int i`: The `for` loop uses an integer counter, which is another constant-size variable.

Even though the function makes two passes over the list (which impacts its **time complexity**, making it O(n)), the **space complexity** is not affected. It only needs a few fixed-size variables to do its job.

Therefore, the auxiliary space complexity is **Θ(1)**.

$Algorithm$

$GetNthNodeFromEnd2(START, NODE)$

$1. SET\ COUNT := 0$

$2. SET\ CURRPTR := START$

$3. Repeat\ while\ CURRPTR \neq NULL, then:$

$\quad SET\ CURRPTR := LINK[CURRPTR]$

$\quad SET\ COUNT := COUNT + 1$

$4. if\ \ NODE < 0\ OR\ \ NODE \geq COUNT, then:$

$\quad Write: ``\ Error: INVALID\ INDEX``$

$\quad RETURN - 1\ and\ EXIT.$

$ELSE$

$\quad SET\ CURRPTR := START$

$\quad Repeat\ for\ I = 0\ to\ (len - node - 1) - 1:$

$\qquad SET\ CURRPTR := LINK[CURRPTR]$

$\quad [END\ OF\ LOOP]$

$\quad Return\ INFO[\ CURRPTR]$

$[END\ OF\ IF\ STRUCTURE]$

$[END\ OF\ ALGORITHM]$